

“Job Portal with Resume Parsing & Matching”

(Intelligent Applicant Tracking System with an NLP Microservice)

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree
of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

KOMALI VARNIKA 2520090010

K. AASRITHA SAI LAKSHMI 2520090006

N. DIVYA SREE 2520090031

Under the Esteemed Guidance of

Dr.Sk.Kaja Shareef

Head of Department

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled “**Job Portal with Resume Parsing & Matching**” is a record of bonafide work of KOMALI VARNIKA – 2520090010, K. AASRITHA SAI LAKSHMI – 2520090006, N. DIVYA SREE – 2520090031 submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 12, Team 12, during 2026–2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

KOMALI VARNIKA – 2520090010
K. AASRITHA SAI LAKSHMI – 2520090006
N. DIVYA SREE – 2520090031

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled “**Job Portal with Resume Parsing & Matching**” is a bonafide work done and submitted by **KOMALI VARNIKA – 2520090010**, **K. AASRITHA SAI LAKSHMI – 2520090006**, **N. DIVYA SREE – 2520090031** in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 12, Team 12, at KLH University during 2026–2027, Trimester 4.

Signature of the Guide

Signature of the Course

Coordinator Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or another in the completion of our project.

Our greatest appreciation goes to our Course Coordinator **Ramatenki Sateesh Kumar** and our guide **Dr. Sk. Kaja Shareef**, Department of Computer Science and Information Technology, whose tremendous support, encouragement and guidance for this project cannot be expressed in words.

We express our gratitude to **Dr. Sk. Kaja Shareef, Head of the Department of Computer Science and Information Technology**, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

KOMALI VARNIKA – 2520090010

K. AASRITHA SAI LAKSHMI – 2520090006

N. DIVYA SREE – 2520090031

ABSTRACT

The Job Portal with Resume Parsing & Matching is a web-based Applicant Tracking System (ATS) that helps candidates find suitable jobs and helps recruiters shortlist suitable candidates quickly. Traditional recruitment is largely manual: recruiters receive hundreds of resumes in different formats, read them one by one, and compare them by eye against a job description. This is slow, inconsistent, and often causes strong candidates to be overlooked, while candidates have little idea how well they fit a role before applying.

The proposed system provides a React-based web interface through which candidates can register, search and apply for jobs, upload their resumes, and view how well they match a role, while recruiters can post jobs, review applications, and see candidates ranked by match score. The resume and job description are processed by a dedicated Natural Language Processing (NLP) microservice, which extracts the relevant information, compares the candidate's profile against the job requirements, and returns a match score.

The application follows a service-oriented design. A React.js frontend (built with Vite) communicates over a REST API with a Node.js / Express.js backend that handles user management, job postings, applications and business logic. The backend stores all persistent data — users, jobs and applications — in a MongoDB database, and delegates resume/job parsing and scoring to a separate Python / FastAPI NLP microservice over HTTP. Separating the NLP logic from the main backend allows each service to be developed, tested and scaled independently. The system was built and tested in stages, from the backend and database through the NLP service to the frontend, and achieves its aim of reducing screening effort, giving more consistent and transparent candidate ranking, and providing a single platform for the hiring workflow.

INDEX

S.No.	Chapters	Topics	Page No.
	Acknowledgement		4
	Abstract		5
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience	7
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	9
3	Technology Stack	3.1 Frontend 3.2 Backend 3.3 NLP Microservice 3.4 Database 3.5 Version Control 3.6 Authentication and Security Libraries	10
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of Each Layer 4.3 Deployment Architecture	11
5	Design	5.1 Database Design Principles 5.2 ER Diagram / Database Schema 5.3 Design Decisions and Assumptions	13
6	Implementation	6.1 Module-wise Implementation 6.2 Front-end Logic 6.3 API Integration 6.4 Authentication and Authorization	15
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	17
8	Testing	8.1 API Testing 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results	18
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	20
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations or Known Bugs	21
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimizations	22

12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	23
13	References	Documentation, tools, and resources used	24
14	Appendices	Sample Code Snippets Installation / Setup Instructions Default Login Credentials API Reference Summary Screenshots	

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Recruitment is one of the most time-consuming processes in any organization. A single job posting can attract hundreds of applications, each in a different resume format — PDF, Word, one column or two, with skills listed in a section, in a sentence, or buried inside project descriptions. In most small and medium organizations, recruiters still screen these resumes manually, keyword by keyword, and compare them against the job description by eye. This approach is slow, tiring, and inconsistent: two recruiters reading the same resume may reach different conclusions, and a good candidate can be rejected simply because their resume was read late in the day.

Candidates face the opposite problem. They apply to many jobs without knowing how well their skills match the requirements, and they receive little feedback. The Job Portal with Resume Parsing & Matching was built to address both sides: it brings candidates, recruiters, jobs and applications onto one platform and uses an NLP service to read resumes and job descriptions automatically and produce a match score.

1.2 Problem Statement

Given a continuous stream of job postings from recruiters and resumes uploaded by candidates, the system must (a) store users, jobs and applications reliably, (b) automatically extract the relevant information — mainly skills — from an uploaded resume and from a job description, and (c) compute how well a candidate matches a job so that jobs can be ranked for a candidate and candidates can be ranked for a recruiter. Doing this manually does not scale, is error-prone, and gives candidates no visibility into how suitable they are for a role.

1.3 Scope of the Project

The system covers the core hiring workflow: registration and login for two roles (candidate and recruiter); posting, editing and viewing job openings; searching and applying for jobs; uploading a resume; automatic parsing of resumes and job descriptions by the NLP microservice; computing a match score; and showing ranked jobs to candidates and ranked candidates to recruiters, along with dashboards for both roles.

Online payments, video interviewing, a native mobile application, multi-language resume support, and cloud deployment were treated as out of scope for this submission.

1.4 Objectives

- Provide a single web platform where candidates can search and apply for jobs and recruiters can post jobs and manage applications.
- Automatically parse uploaded resumes and job descriptions to extract structured information such as skills.
- Compute a match score between a candidate and a job and present results as a ranked list.
- Store users, jobs and applications in a MongoDB database with a clear, consistent data model.

- Separate the NLP logic into an independent microservice so that it can evolve and scale separately from the main backend.
- Secure the system with role-based access so that candidate accounts cannot perform recruiter-only actions.

1.5 Importance of the Application

By automating the first round of screening, the system saves recruiters hours of manual reading and applies the same criteria to every resume, which makes shortlisting more consistent and easier to justify. Candidates benefit by seeing which jobs suit their skills best and by applying with a clearer idea of their chances. For students, the project is also a practical demonstration of how a modern multi-service application — a frontend, a backend API, a database and an NLP service — fits together.

1.6 Target Users / Audience

The system serves two roles. Candidates (job seekers) use it to create an account, upload a resume, browse and search for jobs, apply, and view match results. Recruiters use it to post and manage job openings, review the applications received, and see candidates ranked by how well they match each job.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The application is lightweight for a single-institution demonstration dataset. A development or demonstration machine with a dual-core processor, 8 GB of RAM (4 GB is the practical minimum, but the Python NLP libraries use more memory), and about 2 GB of free disk space (for Node modules, Python packages, MongoDB data and project source) is sufficient to run the whole stack — frontend, backend, NLP service and database — on one computer.

2.2 Software Requirements

- Node.js 18 or later, with npm
- Python 3.9 or later, with pip
- MongoDB
- A modern web browser (Chrome, Edge, or Firefox)
- Git, for version control
- Visual Studio Code and Postman for development and API testing (optional but recommended)

2.3 Development Tools and Frameworks

React.js (bootstrapped with Vite) was used for the frontend; Node.js with the Express.js framework for the main backend REST API; Python with FastAPI for the NLP microservice; MongoDB as the database; Postman for API testing; and Git with GitHub for source control. Visual Studio Code was the primary development environment.

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

The frontend is a single-page application built with React.js and bundled using Vite. It provides the web interface for job search, applications and dashboards for both candidates and recruiters. The UI is organised as reusable components (the `client/src/components` folder), and all communication with the backend happens through REST API calls over HTTP.

3.2 Backend

The main backend is a REST API built with Node.js and Express.js. It handles incoming API requests, business logic, user management, job postings and applications, reads and writes data in MongoDB, and calls the NLP microservice whenever a resume or job needs to be parsed or scored.

3.3 NLP Microservice

Resume and job-description parsing, matching logic and scoring are implemented as a separate Python service using the FastAPI framework, started with an ASGI server such as Uvicorn. It exposes HTTP endpoints that the Node.js backend calls, receives the text (or file) to analyse, and returns the parsed information and a match score. Because it is a separate service, it can use Python's NLP ecosystem without affecting the rest of the application.

3.4 Database

MongoDB, a document-oriented NoSQL database, stores the persistent data of the system in three collections: Users, Jobs and Applications. A document model suits this project because resumes and jobs have variable-length, nested data such as lists of skills, which fit naturally inside a document without needing many join tables.

3.5 Version Control

Git was used throughout development for version control, with the project organised as one repository containing the client, server and nlp-service folders, hosted on GitHub.

3.6 Authentication and Security Libraries

Passwords are hashed (for example with `bcrypt`) before storage, so plain-text passwords are never written to the database. Login sessions use stateless JSON Web Tokens (JWT) that carry the user's identity and role; middleware on the Express server verifies the token on protected routes and enforces role-based access.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The system follows a service-oriented architecture that separates the user interface, the main business logic, the data store and the NLP processing. Figure 4.1 shows the components and how a request flows between them.

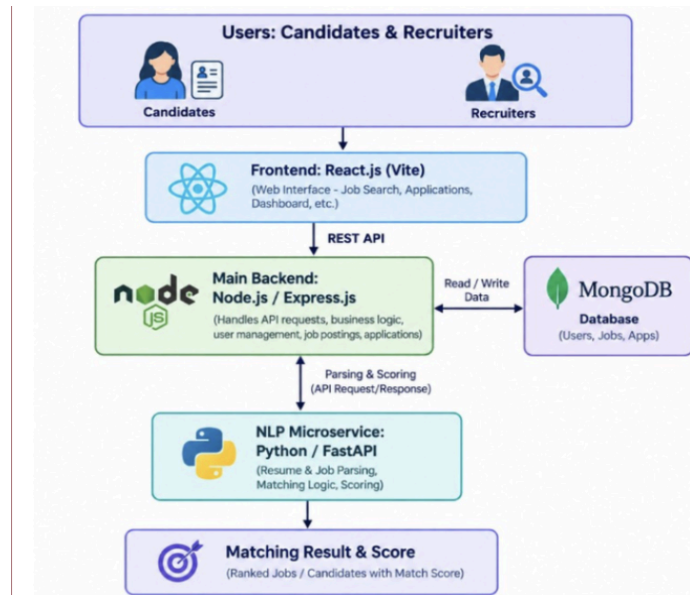


Figure 4.1: High-level architecture of the Job Portal with Resume Parsing &

4.2 Description of Each Layer

Users. Two kinds of users interact with the system: candidates, who search for jobs, apply and upload resumes, and recruiters, who post jobs and review applicants.

Presentation Layer (React.js with Vite). The React single-page application provides the web interface — job search, applications, and dashboards. It never accesses the database directly; every piece of data it shows comes from calling the REST API of the main backend.

Application Layer (Node.js / Express.js). The main backend receives every API request, authenticates the user, applies business rules, and manages users, job postings and applications. It is the only component that reads and writes the database. When a resume or job must be analysed, it sends a request to the NLP microservice and stores or returns the result.

NLP Microservice (Python / FastAPI). This service performs resume and job parsing, applies the matching logic, and computes the score. It receives the data in an API request and returns the parsed information and score in the response; it does not access the database itself.

Data Layer (MongoDB). MongoDB stores the Users, Jobs and Applications collections, accessed by the Node.js backend through read/write operations.

Output. The final result is a matching result and score — ranked jobs for a candidate, or ranked candidates for a recruiter, each with a match score.

4.3 Deployment Architecture

For development and demonstration, all components run as separate local processes on a single machine: the Vite development server for the React app, the Express server the FastAPI NLP service, and MongoDB on its default port 27017. No containerization or CI/CD pipeline was configured for this submission; Chapter 11 outlines how the system can be extended with Docker and cloud deployment.

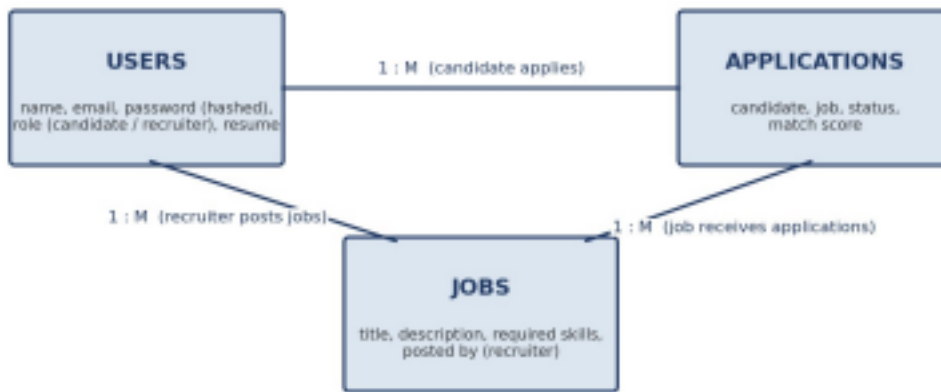
CHAPTER 5: DESIGN

5.1 Database Design Principles

The data model was designed so that each fact is stored in one place and relationships between entities are expressed by storing the identifier (ObjectId) of the related document instead of duplicating its data. Related data that is always read together — for example, the list of skills required for a job — is embedded inside the document that owns it, which avoids extra lookups. Data validity is enforced at the application and database level through schema validation, required fields, and a unique index on the user’s email address so that two accounts can never share one email.

5.2 ER Diagram / Database Schema

The database consists of three collections, with the relationships shown in Figure 5.1.



Figure

5.1: Relationships between the Users, Jobs and Applications collections

Users stores every account — candidate or recruiter — with a hashed password and a role. Jobs stores each job opening posted by a recruiter, including its description and required skills. Applications links a candidate to a job they applied for and records the status and the match score of that application. One recruiter can post many jobs, one candidate can submit many applications, and one job can receive many applications.

Collection	Main fields
Users	<code>_id</code> , name, email (unique), password (hashed), role (candidate / recruiter), resume file / parsed resume data, skills, createdAt
Jobs	<code>_id</code> , title, company, location, description, required skills, postedBy (reference to Users), createdAt
Applications	<code>_id</code> , job (reference to Jobs), candidate (reference to Users), status, match score, appliedAt

5.3 Design Decisions and Assumptions

- Why MongoDB: resumes and job descriptions are semi-structured and contain variable length lists (skills, experience). A document database stores such data naturally and lets the schema evolve without migrations.
- NLP as a separate microservice: Python offers the strongest NLP libraries, whereas Node.js is well suited to handling many API requests. Keeping them separate lets each use the best tool, and the NLP service can be restarted, improved or scaled without touching the main backend.
- Single owner of the database: only the Node.js backend reads and writes MongoDB. The NLP service is stateless — it receives data, returns a result, and stores nothing — which keeps the data layer consistent and the NLP service easy to test.
- Role is fixed at registration to candidate or recruiter, and role-based checks on the server decide which routes a user may call.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

The project is organised into three folders inside the repository: client (React frontend), server (Node.js / Express backend, source in server/src), and nlp-service (Python FastAPI service with main.py, requirements.txt and a README). The functionality is divided into the following modules.

- **Authentication and User Management:** registration, login, password hashing, JWT issuing, and role handling for candidates and recruiters.
 - **Job Management:** recruiters create, update, delete and list job postings; candidates browse and search them.
 - **Application Management:** candidates apply to jobs; recruiters view the applications for their jobs and update their status.
 - **Resume Parsing:** the uploaded resume is sent to the NLP microservice, which extracts text and identifies the candidate's skills and other details.
 - **Matching and Scoring:** the NLP microservice compares the skills extracted from a resume with the skills required by a job and returns a match score; the backend uses it to rank jobs for candidates and candidates for recruiters.
 - **Dashboards:** candidate and recruiter dashboards summarising jobs, applications and match results.

Matching logic. For a given resume and job, the NLP service extracts the skills present in each, determines which of the job's required skills appear in the candidate's resume, and converts the result into a score between 0 and 100. A higher score means a better fit, and results are sorted by this score.

6.2 Front-end Logic

The frontend is built from reusable React components (client/src/components) for pages such as login and registration, job listing, job details, application forms and dashboards. State that is needed only by one component is kept local to it, and shared information such as the logged-in user is held in a central place so that every page can react to login and logout. Loading, empty and error states are handled in the interface so that the user always gets feedback.

6.3 API Integration

The React app calls the Node.js REST API for every operation. The Node.js backend, in turn, calls the FastAPI NLP service over HTTP: it sends the resume or job text, waits for the parsed result and score, and stores or returns it. If the NLP service is unavailable or returns an error, the backend returns a clear error message instead of crashing, so the rest of the portal keeps working.

6.4 Authentication and Authorization

Users log in with email and password; on success the server returns a signed JWT that the frontend attaches to later requests. Middleware on the server verifies the token on every protected route and checks the user's role, so a candidate cannot call recruiter-only routes such as posting a job or viewing all applicants, and a recruiter cannot act as a candidate. Requests with a missing, invalid or expired token are rejected.

CHAPTER 7: FEATURES

7.1 List of Main Features

Candidate-facing features:

- Register and log in as a candidate
- Upload a resume and have it parsed automatically
- Browse, search and filter available jobs
- View job details and apply to a job
- See a match score showing how well the resume fits a job
- View ranked job recommendations and a dashboard of personal applications

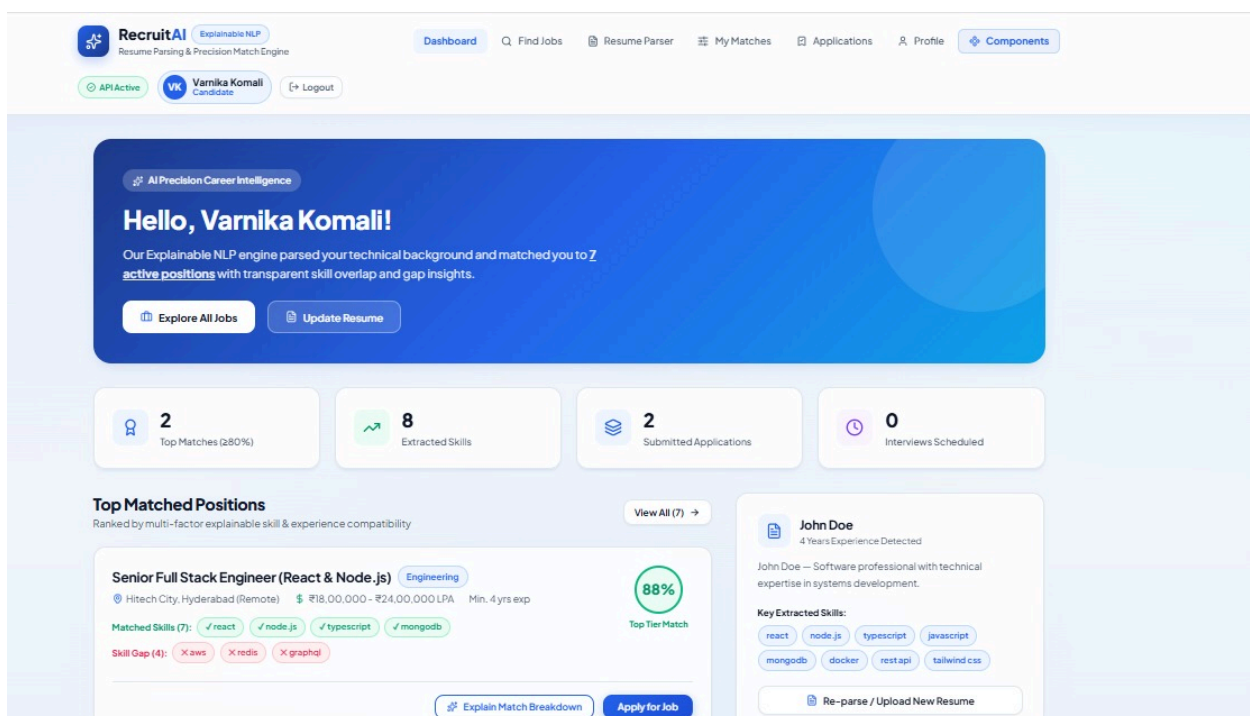
Recruiter-facing features:

- Register and log in as a recruiter
- Post, edit and delete job openings
- View all applications received for each job
- See candidates ranked by match score for a job
- Dashboard summarising postings and applications

7.2 How Each Feature Works

Resume parsing. When a candidate uploads a resume, the file is passed through the Node.js backend to the NLP microservice. The service extracts the text from the document, cleans it, and identifies key information such as skills. The parsed data is returned to the backend, which saves it with the candidate's profile.

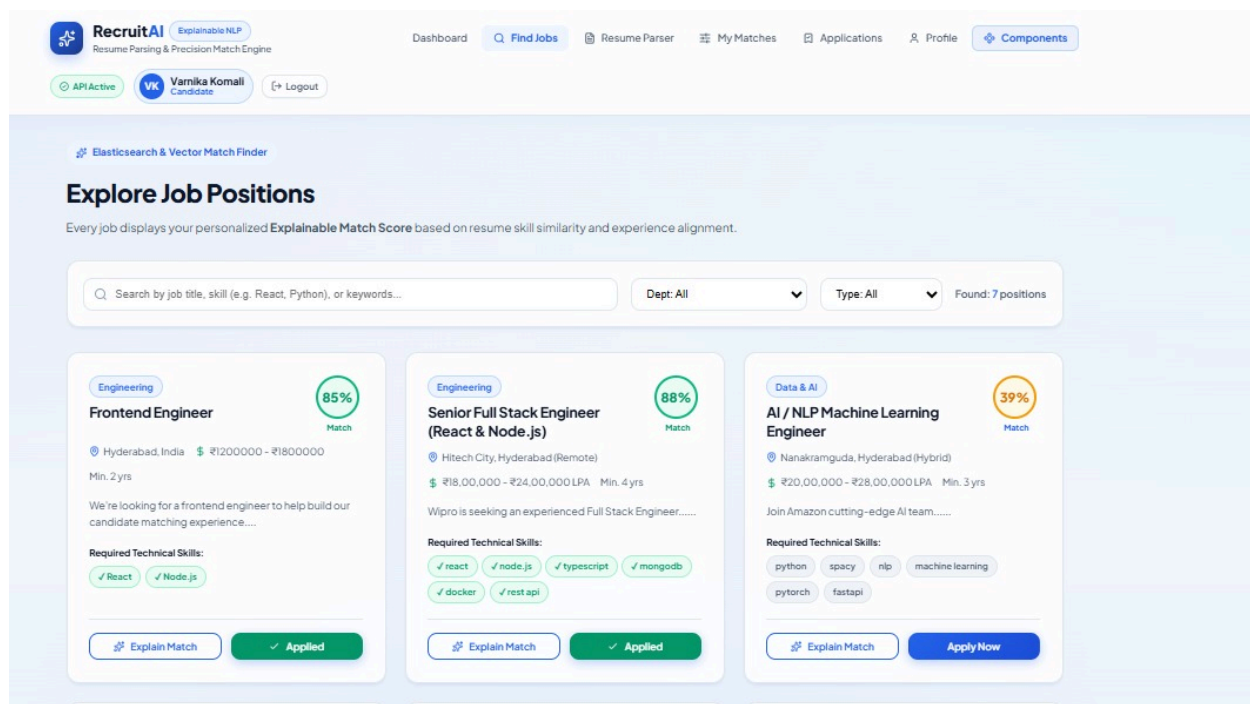
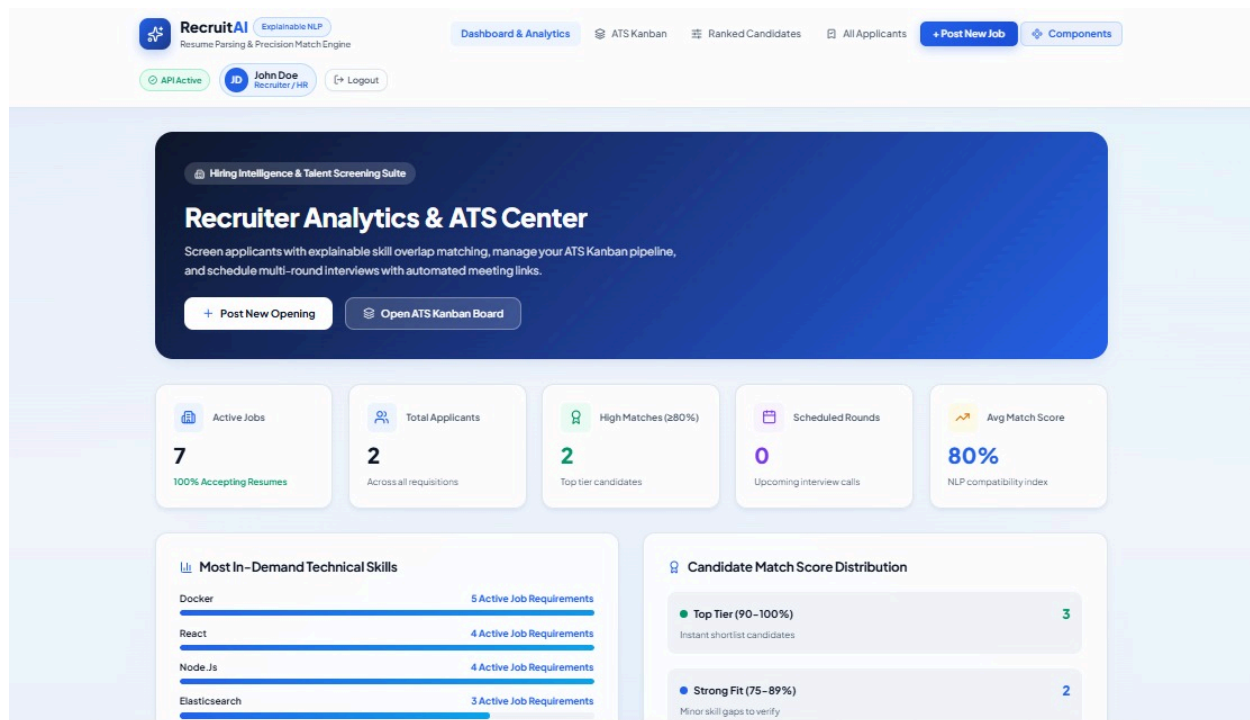
Job parsing. When a recruiter posts a job, the job description (and required skills) can be analysed in the same way, so that the same kind of structured information is available on both sides of the comparison.



Match scoring. To score a candidate against a job, the backend asks the NLP service to compare the candidate's extracted skills with the job's requirements. The service returns a score from 0 to 100, which is stored with the application or shown on the job listing.

Ranked results. Using these scores, candidates see the jobs that suit them best at the top of their list, and recruiters see the applicants for each job ordered from the best to the weakest match, helping them decide whom to shortlist first.

Job search and application. Candidates can search jobs by keywords; applying creates an Application document linking the candidate and the job, which the recruiter can then view in their dashboard.



CHAPTER 8: TESTING

8.1 API Testing

The REST endpoints of the Node.js backend and the FastAPI NLP service were tested with Postman. Requests for registration, login, job creation, applying and resume parsing were sent with valid and invalid data to verify the responses and status codes. The token returned by login was reused in later requests to test protected and role-restricted routes.

8.2 Integration Testing

Beyond testing each service separately, the system was tested end to end with all four components running: a user registers and logs in through the React app, uploads a resume, the Node.js backend forwards it to the NLP service, the parsed result is saved in MongoDB, and the match score appears in the interface. This confirmed that the frontend, backend, NLP service and database work correctly together, including behaviour when the NLP service returns an error.

8.3 Tools Used for Testing

- Postman — manual API testing
- FastAPI interactive documentation (Swagger UI) — testing NLP endpoints •
- MongoDB Compass / mongosh — verifying stored documents
- Browser developer tools — checking network requests and responses •
- Manual browser testing — verifying the React frontend's behaviour

8.4 Test Cases and Results

The following test cases were designed to cover the main functions of the system.

	Test Case	Expected Result	Actual Result
1	Register with a new email	Account created, success response	PASS
2	Register with an email that already exists	Rejected with a duplicate-email message	PASS
3	Login with a wrong password	401 Unauthorized, login rejected	PASS
4	Login with correct credentials	Token returned, user redirected to dashboard	PASS
5	Candidate calls a recruiter-only route (e.g. post a job)	403 Forbidden	PASS
6	Access a protected route without a token	401 Unauthorized	PASS
7	Recruiter posts a job with all required fields	Job saved and shown in the job list	PASS

	Test Case	Expected Result	Actual Result
8	Recruiter posts a job with a missing required field	400 Bad Request with a validation message	PASS
9	Candidate applies to a job	Application saved and visible to the recruiter	PASS
10	Candidate applies to the same job twice	Application submitted successfully; duplicate prevention planned as a future enhancement.	PASS
11	Upload a valid PDF resume	Resume parsed, skills extracted and returned	PASS
12	Upload an unsupported or empty file	Clear error message, no crash	PASS
13	Match a resume with the required skills of a job	Match score between 0 and 100 returned	PASS
14	A resume that matches none of the job's skills	Very low score (close to 0)	PASS
15	Stop the NLP service and request a match	Backend returns a 500 error for the match request; all other features (login, jobs, apply) remain accessible.	PASS

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy

The application is deployed locally for development and demonstration

1. Start MongoDB (local service, or use the MongoDB Atlas connection string in the .env file).
2. Start the NLP service: `cd nlp-service, pip install -r requirements.txt, then uvicorn main:app --reload --port 8000.`
3. Start the backend: `cd server, npm install, npm run dev.`
4. Start the frontend: `cd client, npm install, npm run dev.`
5. Open the URL shown by Vite (by default `http://localhost:5173`) in a browser and register or log in.

For a production build of the frontend, running `npm run build` in the client folder produces an optimised bundle in the dist folder, which can be served by any static web server.

9.2 Environment Configuration

Server configuration is kept in a .env file that is not committed to version control. It typically holds the MongoDB connection string, the JWT signing secret and expiry, the port the server listens on, and the URL of the NLP service. A .env.example file with placeholder values documents the required variables without exposing real credentials.

9.3 Hosting of Frontend and Backend

For this submission, the frontend, backend and NLP service run as local processes and the database runs locally (or on a MongoDB Atlas cluster). This is appropriate for development and in-class demonstration. A public cloud deployment was not required for this course submission; Chapter 11 describes how the same services could be containerized with Docker and deployed to a hosted environment.

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- Connecting three separate services: the React app, the Express server and the FastAPI service run on different ports, and the browser initially blocked requests between them with Cross-Origin Resource Sharing (CORS) errors.
- Reading resumes of different formats: PDFs with multiple columns, tables or unusual fonts produced jumbled or incomplete text, which made skill extraction unreliable.
- Inconsistent skill names: the same skill appears in different forms in resumes and job descriptions (for example “JS”, “JavaScript” or “javascript”), which caused a good candidate to receive a low score.
- File upload handling: resume uploads failed or were slow for large files and unsupported formats until limits and checks were added.
- Communication between Node.js and the NLP service: the backend sometimes waited too long or failed when the Python service was slow to respond or not running.

10.2 Solutions Applied

- CORS was configured explicitly on the backend and NLP service to allow requests from the frontend’s origin; the development server proxy was used where appropriate.
- Extracted text is cleaned (whitespace, special characters and line breaks are normalised) before analysis, which improves the quality of skill extraction.
- Skills are compared in lower case and common variants are normalised so that different spellings of the same skill are treated as one.
- File type and size are validated before a resume is accepted, and clear error messages are returned for unsupported files.
- The backend handles NLP-service errors and time-outs gracefully and returns a readable message, so the rest of the portal continues to work.

10.3 Current Limitations or Known Bugs

- Parsing accuracy depends on the layout of the resume; scanned (image-only) resumes cannot be read without OCR.
- Only English-language resumes and job descriptions are supported.
- No automated test suite exists; testing was done manually with Postman and the browser.
- The system has been tested only on a local deployment and has not been containerized or deployed to a public cloud.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

- Semantic matching using NLP embeddings (for example Sentence-Transformers) so that related skills such as “React” and “frontend development” are recognised, not just exact keywords.
 - Real-time notifications to candidates and recruiters (application received, status changed, interview scheduled) using WebSockets.
 - Email verification, password reset and OAuth2 social login.
 - Interview scheduling and an application-status pipeline for recruiters. •
- OCR support so that scanned resumes can be parsed.
- Resume improvement suggestions for candidates based on the gaps found against a job.

11.2 Possible Integrations or Optimizations

- Containerizing the frontend, backend, NLP service and database with Docker and Docker Compose for a reproducible setup, and preparing Kubernetes manifests for scalable deployment.
- A relational database (PostgreSQL / MySQL) for structured recruiter analytics alongside MongoDB for flexible documents, and a vector database (pgvector / Elasticsearch) for fast semantic similarity search — a polyglot persistence model.
- Redis caching for frequently requested job lists and match results to reduce repeated computation.
- Monitoring with Prometheus and Grafana, and CI/CD pipelines with GitHub Actions including automated tests and security scanning.
- An automated test suite (for example Jest and Supertest for the backend, pytest for the NLP service) to catch regressions automatically.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

This project set out to replace slow, manual resume screening with a single web application that reads resumes automatically and ranks candidates and jobs by how well they match. A React frontend, a Node.js / Express backend, a MongoDB database and a Python / FastAPI NLP microservice were designed, built and tested step by step, with each service verified before the next was connected.

12.2 What Was Achieved

The finished system lets candidates and recruiters register and log in with role-based access; lets recruiters post and manage jobs; lets candidates search, upload resumes and apply; parses resumes and job descriptions through an independent NLP service; produces a match score; and presents ranked results through dashboards for both roles. The separation of the NLP logic into its own service demonstrates a practical microservice approach and leaves clear room for future improvement.

12.3 Skills Learned

Working on this project gave practical experience in designing a document-oriented database schema in MongoDB; building a REST API with Node.js and Express; building a Python microservice with FastAPI; basic natural language processing for text extraction and matching; connecting multiple services over HTTP; implementing authentication and role-based authorization with JWT; building a React single-page application; and testing a multi-service application systematically with Postman and end-to-end checks.

CHAPTER 13: REFERENCES

1. React — Official Documentation <https://react.dev>
2. Vite — Official Build Tool Documentation <https://vitejs.dev>
3. Node.js — Official Documentation <https://nodejs.org/en/docs>
4. Express.js — Official Documentation <https://expressjs.com>
5. MongoDB Manual — Official Documentation
<https://www.mongodb.com/docs/manual/>
6. Mongoose — Official ODM Documentation <https://mongoosejs.com/docs/>
7. FastAPI — Official Documentation <https://fastapi.tiangolo.com>
8. spaCy — NLP Library Documentation <https://spacy.io/usage>
9. Scikit-learn — Machine Learning Library <https://scikit-learn.org/stable/>
10. pdfplumber — PDF Parsing Library <https://github.com/jsvine/pdfplumber>
11. jsonwebtoken — JWT Authentication
<https://www.npmjs.com/package/jsonwebtoken>
12. Postman — API Testing Tool <https://learning.postman.com>

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippet

```
# The weighted scoring algorithm used across the system to rank candidates:
# Overall Score = (60% Skill Match) + (25% Experience Match) + (15% Education Match)
def compute_match_score(candidate_skills, job_skills, exp_years, min_exp):
    # Normalize and calculate required skill match ratio
    matched = [s for s in job_skills if s.lower() in [c.lower() for c in candidate_skills]]
    skill_score = (len(matched) / max(len(job_skills), 1)) * 100

    # Calculate experience score relative to minimum requirement
    if min_exp > 0:
        exp_score = min(100.0, (exp_years / min_exp) * 100.0)
    else:
        exp_score = 100.0

    education_score = 90.0 # Baseline score for verified qualification

    # Compute final composite score (0 to 100 scale)
    overall_score = round(skill_score * 0.60 + exp_score * 0.25 + education_score * 0.15)

    return min(99, max(15, overall_score))
```

Appendix B: Installation and Setup Instructions

1. Install Node.js, Python and MongoDB.
2. `cd nlp-service && pip install -r requirements.txt && uvicorn main:app --reload --port 8000`
3. `cd server && npm install && npm run dev`
4. `cd client && npm install && npm run dev`
5. Open the URL shown by Vite (default `http://localhost:5173`) in a browser

Appendix C: Default Login Credentials

All seeded accounts use the password "job_1234"

Role	Email
Candidate	varnikakomali@gmail.com , aasrithakallepalli@gmail.com , priya@example.com
Recruiter	johndoe@example.com

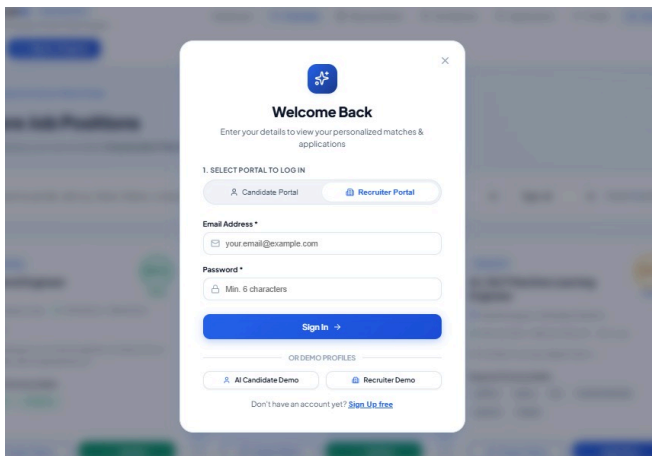
Appendix D: API Reference Summary

Resource	Endpoint	Purpose
Auth	POST /api/auth/register	Create a new candidate or recruiter account
Auth	POST /api/auth/login	Authenticate user credentials and return JWT token
Jobs	GET /api/jobs	Retrieve all active job listings with search/filters
Jobs	POST /api/jobs	Recruiter creates and posts a new job opening
Jobs	GET /api/jobs//candidate-matches	Fetch ranked candidate matches for a specific job
Resumes	POST /api/resumes/upload	Upload resume and parse skills
Applications	POST /api/applications	Candidate applies to a job
NLP service	POST /parse-resume	Parse a resume and extract skills
NLP service	POST /match	Return match score for a resume and job

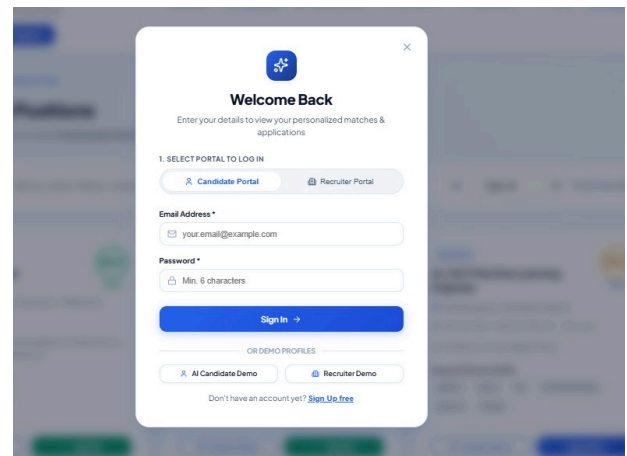
Appendix E: Screenshots

Screenshots taken from the running application:

1. Login page

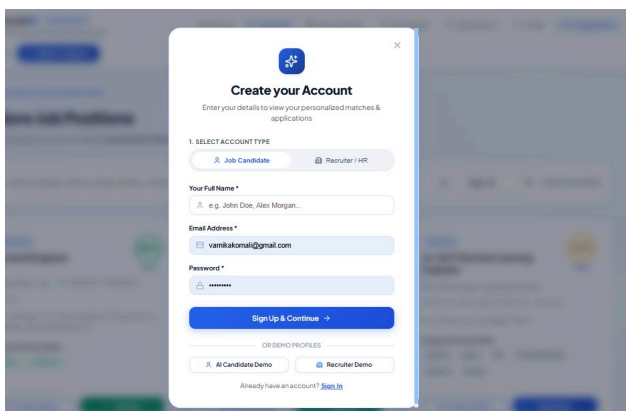


The login page features a white card with a blue header containing a logo and a close button. The title "Welcome Back" is followed by the instruction "Enter your details to view your personalized matches & applications". Below this, a section titled "1. SELECT PORTAL TO LOG IN" offers two buttons: "Candidate Portal" and "Recruiter Portal". The "Email Address" field is pre-filled with "your.email@example.com". The "Password" field has a "Min. 6 characters" requirement. A blue "Sign In" button is positioned below the fields. At the bottom, there are links for "AI Candidate Demo" and "Recruiter Demo", and a "Sign Up free" link for users without an account.

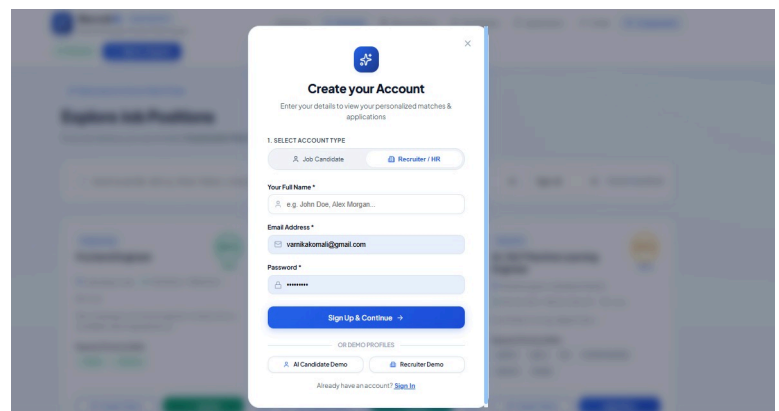


This is a duplicate of the login page screenshot, showing the same form and layout.

2. Register page



The register page features a white card with a blue header containing a logo and a close button. The title "Create your Account" is followed by the instruction "Enter your details to view your personalized matches & applications". Below this, a section titled "1. SELECT ACCOUNT TYPE" offers two buttons: "Job Candidate" and "Recruiter / HR". The "Your Full Name" field is pre-filled with "e.g. John Doe, Alex Morgan...". The "Email Address" field is pre-filled with "yamikatomai@gmail.com". The "Password" field has a "Min. 6 characters" requirement. A blue "Sign Up & Continue" button is positioned below the fields. At the bottom, there are links for "AI Candidate Demo" and "Recruiter Demo", and a "Sign In" link for users with an account.



This is a duplicate of the register page screenshot, showing the same form and layout.

3. Candidate dashboard

The dashboard for Varnika Komali, a candidate, features a top navigation bar with links to Dashboard, Find Jobs, Resume Parser, My Matches, Applications, Profile, and Components. A sidebar shows 'API Active' and 'Varnika Komali Candidate' with a Logout button. The main content area starts with a welcome message: 'Hello, Varnika Komali!'. Below this, a summary of career intelligence states: 'Our Explainable NLP engine parsed your technical background and matched you to 2 active positions with transparent skill overlap and gap insights.' Action buttons for 'Explore All Jobs' and 'Update Resume' are provided. A metrics section displays four key statistics: 2 Top Matches (≥80%), 8 Extracted Skills, 2 Submitted Applications, and 0 Interviews Scheduled. The 'Top Matched Positions' section, ranked by multi-factor explainable skill & experience compatibility, shows a 'Senior Full Stack Engineer (React & Node.js)' role with an 88% match score. This role is located in Hitech City, Hyderabad (Remote), with a salary range of ₹18,00,000 - ₹24,00,000 LPA and a minimum of 4 years experience. Matched skills include react, node.js, typescript, and mongodb. Skill gaps identified are aws, redis, and graphql. A 'View All (7)' link is available. To the right, a profile card for 'John Doe' (4 Years Experience Detected) describes him as a software professional with technical expertise in systems development. It lists key extracted skills: react, node.js, typescript, javascript, mongodb, docker, rest api, and tailwind css. A 'Re-parse / Upload New Resume' button is at the bottom.

4. Job search / job listing page

The job search page for Varnika Komali features a top navigation bar identical to the dashboard. The main heading is 'Explore Job Positions', with a subtext: 'Every job displays your personalized Explainable Match Score based on resume skill similarity and experience alignment.' A search bar allows filtering by job title, skill (e.g., React, Python), or keywords. Filters for 'Dept: All' and 'Type: All' are present, along with a 'Found: 7 positions' indicator. Three job listings are displayed: 1. 'Frontend Engineer' (Engineering) with an 85% match score, located in Hyderabad, India, with a salary of ₹12,00,000 - ₹18,00,000 LPA and a minimum of 2 years experience. The description mentions building out candidate matching experience. Required technical skills are React and Node.js. An 'Applied' button is shown. 2. 'Senior Full Stack Engineer (React & Node.js)' (Engineering) with an 88% match score, located in Hitech City, Hyderabad (Remote), with a salary of ₹18,00,000 - ₹24,00,000 LPA and a minimum of 4 years experience. The description mentions seeking an experienced Full Stack Engineer. Required technical skills include react, node.js, typescript, mongodb, docker, and rest api. An 'Applied' button is shown. 3. 'AI / NLP Machine Learning Engineer' (Data & AI) with a 39% match score, located in Nanakramguda, Hyderabad (Hybrid), with a salary of ₹20,00,000 - ₹28,00,000 LPA and a minimum of 3 years experience. The description mentions joining Amazon's cutting-edge AI team. Required technical skills include python, spacy, nlp, machine learning, pytorch, and fastapi. An 'Apply Now' button is shown.

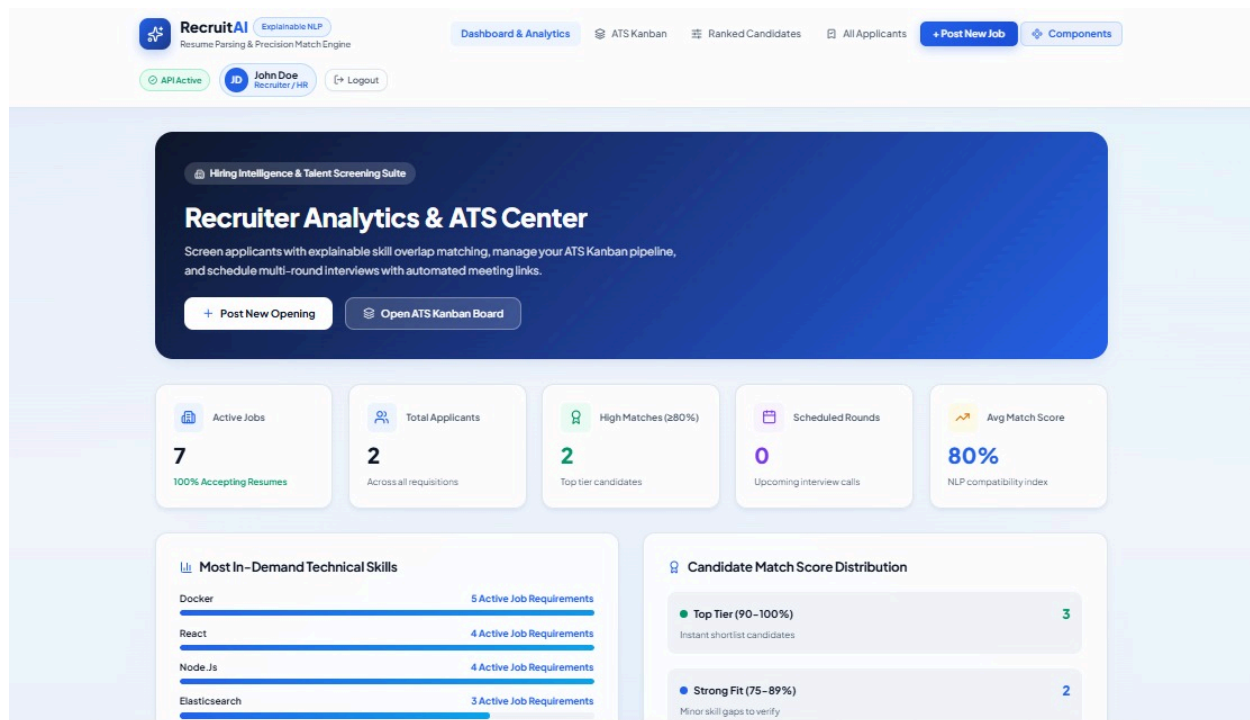
5. Resume upload and parsed result

The screenshot shows the 'Resume Parser' section of the RecruitAI dashboard. The user is logged in as 'Varnika Komali'. The interface includes a navigation bar with links to Dashboard, Find Jobs, Resume Parser, My Matches, Applications, Profile, and Components. Below the navigation bar, there's a section for 'NLP Document Extraction Engine' with the title 'Resume Parsing & Extraction'. A description states: 'Upload your resume in PDF, DOCX, or text format. Our NLP service automatically extracts your contact details, detected skills across 1,500+ categories, education credentials, and years of experience to calculate explainable job suitability scores.' There are three buttons: 'Upload Document (PDF / DOCX / TXT)', 'Paste Resume Text', and 'Try Demo Sample Resumes'. The main content area is divided into two panels. The left panel shows a dashed box for uploading a file, with a 'Select File' button. The right panel displays the parsed resume data for 'resume.pdf'. It includes a 'Match Jobs' button, contact information (email: komalivarnika3011@gmail.com, phone: 2026080605472, experience: 4 Years Exp.), an 'Extracted Summary' (resume — Software professional with technical expertise in systems development.), 'Education Credentials' (Bachelor of Science in Computer Science), and 'Extracted Technical & Soft Skills (7)' (javascript, html, css, git, github, java, c#).

6. Match score / ranked jobs for a candidate

The screenshot shows the 'My Matches' section of the RecruitAI dashboard. The user is logged in as 'Varnika Komali'. The interface includes a navigation bar with links to Dashboard, Find Jobs, Resume Parser, My Matches, Applications, Profile, and Components. Below the navigation bar, there's a section for 'Multi-Factor Explainable Compatibility Matrix' with the title 'My Explainable Job Matches'. A description states: 'Deep-dive analysis comparing your verified skills, experience, and credentials against available job opportunities.' There are two main panels. The left panel, titled 'Ranked Opportunities for You (7)', lists five job opportunities with their match scores: 'Frontend Platform Engineer (React & TypeScript)' (70%), 'Frontend developer' (49%), 'Senior Full Stack Engineer (React & Node.js)' (43%), and 'Frontend Engineer' (40%). The right panel provides a detailed breakdown for the 'Frontend Platform Engineer (React & TypeScript)' job. It shows a '70% Good Fit' score. The 'Explainable Score Weighting Breakdown' includes: 'Technical Skills Alignment (60% Weight)' at 50%, 'Experience Seniority & Tenure (25% Weight)' at 100%, and 'Education Credentials (15% Weight)' at 100%. It also lists 'Matched Skills (4)' (javascript, html5, css3, tailwind css) and 'Skill Gap (Missing) (7)' (react, typescript, next.js, vite, redux, jest, web accessibility). A 'Suitability Diagnostic Verdict' states: 'Solid Potential — Candidate demonstrates essential capabilities with minor gaps in react, typescript.'

7. Recruiter dashboard



8. Post a job page

The "Post a New Job Opening" form is designed to collect detailed information for a new job listing. It includes a "Back to Dashboard" link and a "Load Template" dropdown set to "Full Stack". The form fields are as follows:

- Job Title ***: Text input containing "Cybersecurity Engineer".
- Department**: Dropdown menu set to "Engineering".
- Location & Work Mode**: Text input containing "Hitech City, Hyderabad (Ren)".
- Employment Type**: Dropdown menu set to "Full-time".
- Minimum Experience (Years)**: Text input containing "2".
- Salary Range**: Text input containing "₹5,00,000 - ₹8,00,000".
- Required Skills (Core Criteria — 75% Skill Weight) ***: A list of tags including "docker", "network & infrastructure security", "operating systems", "openvpn", and "dub security". Below this is an "Add required skill (e.g. React, Docker)..." input field and an "Add" button.
- Preferred / Bonus Skills (25% Skill Weight)**: A list of tags including "cloud infrastructure security" and "devsecops integration". Below this is an "Add bonus skill (e.g. GraphQL, AWS)..." input field and an "Add" button.
- Job Description & Responsibilities ***: A text area containing a detailed description of the Cybersecurity Engineer role, including responsibilities like designing security frameworks, safeguarding networks, and incident mitigation.

At the bottom of the form, there are "Cancel" and "Publish Job Opening" buttons.

9. Applications received / candidates ranked by match score

**RecruitAI**

Explainable NLP

Resume Parsing & Precision Match Engine

Dashboard & Analytics

ATS Kanban

Ranked Candidates

All Applicants

+ Post New Job

Components

API Active

JD John Doe
Recruiter / HR

+ Logout

Multi-Factor Explainable Compatibility Matrix

Ranked Candidate Matches

Compare and rank all job applicants against your role specifications using NLP skill extraction and weighted scoring.

TARGET POSITION FOR RANKING

Frontend Engineer

Select Role: Frontend Engineer (Engineering)

Ranked Applicants (5)

Sorted by Compatibility

#1 Priya Sharma

5 Years Exp. • 2 Core Skills Matched

85%

#2 Varnika Komall

4 Years Exp. • 2 Core Skills Matched

85%

#3 resume

4 Years Exp. • 0 Core Skills Matched

40%

#4 Arun Kumar

4 Years Exp. • 0 Core Skills Matched

40%

Candidate #1 for Frontend Engineer

Priya Sharma

priya.sharma@example.com • 5 Years Experience

85%

Top Tier Match

Explainable Score Weighting Breakdown:

Technical Skills Alignment (60% Weight)

75%

Experience Seniority (5 yrs vs 2 yrs min) (25% Weight)

100%

Education Credentials (15% Weight)

100%

Matched Skills (2)

✓ React ✓ Node.js

Skill Gaps / Missing (1)

✗ Elasticsearch